

Agents, Forces and Voronoi Distance

Veronica Sgamuffa

June 2026

1 Introduction

This project simulates the movement of multiple agents within an area. Each agent moves toward the centroid of its Voronoi cell and simultaneously interacts with other agents connected via a graph so that, due to attractive forces, the connection between them is not lost. Agents are initialized to random positions near the center. A random Erdos-Rényi graph is generated where each pair of nodes has probability p of being connected (neighbors are defined via a symmetric adjacency matrix); each agent knows only the neighbors connected to it. To run the project, written in C++ with the use of the OpenMP framework for functions working in parallel, a pc was used that has the specifications shown in the following table:

CPU	AMD Ryzen 7 7730U
Core/Threads	8/16
RAM memory	16GB

2 Implementation

The project consists of a computational engine developed in C++ and a visualization system implemented in Python.

The geometric and kinematic logic of the simulation is fully implemented in the main.cpp file, starting with the Vector2D structure. This construct handles two-dimensional Cartesian coordinates and overloads the fundamental algebraic operators. This choice was made to perform direct vector operations, optimizing performance and eliminating the computational

overhead typical of calls to external functions. The structure also includes the method for calculating the Euclidean norm, an essential requirement for deriving vector magnitudes and the mutual distances between the agents in the group.

The ‘dot_product’ function computes the dot product between vectors, providing the algebraic basis for calculating orthogonal projections and defining subspaces. The ‘clip_polygon’ function relies directly on it, implementing a variant of the Sutherland-Hodgman algorithm for clipping convex polygons. This module takes as input the vertices of the current polygon boundary, a midpoint, and a normal vector, which define a linear separating hyperplane. Using the dot product, the function evaluates whether the vertices are included in or excluded from the admissible subspace. When a segment intersects the boundary line, the algorithm analytically calculates the linear interpolation coefficient to determine the exact coordinate of the new intersection, returning a polygon limited to only the valid spatial portion.

Once the geometric cell has been isolated, the task of determining the agent’s equilibrium configuration is delegated to the ‘polygon_centroid’ function. This module applies the Gauss area formula (shoelace formula) by performing a linear scan of adjacent vertex pairs. This computation allows for the accumulation of cross-products of coordinates to quantify the surface area and, simultaneously, to weight the vertices in order to obtain the first-order static moments necessary for calculating the center of mass. To ensure numerical robustness and avoid singularities resulting from division by zero, the function introduces an infinitesimal tolerance check on the area, setting a

fallback coordinate in the event that the cell tends toward a degenerate configuration.

The synthesis of these geometric operations takes place in the `compute_exact_voronoi_centroids_seq` function, which is responsible for the sequential generation of the Voronoi tessellation. For each agent in the system, the module defines an initial convex domain corresponding to the physical boundaries of the simulation area. Subsequently, a nested double loop evaluates the mutual spatial position relative to the other competing agents: for each pair, the median separation axis is calculated, converted into a clipping constraint, and iteratively passed to the `clip_polygon` function. This process progressively reduces the extent of the polygon until the agent's exact Voronoi cell is isolated, from which the geometric centroid is finally extracted and returned.

The parallel version is implemented in the function `compute_exact_voronoi_centroids_omp`. This variant inherits the geometric logic of the sequential module but modifies its execution architecture using OpenMP compilation directives. Applying the parallelization construct to the outermost for loop distributes the calculation of Voronoi cells across the various hardware threads. Through a static scheduling policy, the computational load associated with individual agents is distributed evenly across the CPU cores, minimizing synchronization overhead and maximizing the system's computational efficiency.

To prevent the occurrence of isolated nodes (agents with no connections) a phenomenon typical of the pure Erdős-Rényi stochastic model—the `is_connected` function checks the network's global connectivity: starting from an arbitrary root node, the function explores the link topology, recording the reached nodes in a Boolean vector. This routine allows for the detection and elimination of graphs fragmented into disconnected components (generation continues until a single global network is obtained).

The topological verification is integrated into the `generated_connected_graph` function, which handles the stochastic generation of the group network. Using the Mersenne Twister pseudorandom number generator and a uniform statistical distribution, the module assigns a communication link between pairs of nodes

based on a fixed interaction probability. The function operates within an iterative loop that repeatedly calls the verification module, accepting and saving the final adjacency matrix to disk only when the existence of a single globally connected component is mathematically guaranteed.

The main function coordinates the entire application workflow, which is divided into the initialization, sequential execution, and parallel execution phases. During the setup phase, the program defines the parameters of the spatial domain and distributes the initial positions of the agents (`n_agents`) around the center of the map, constraining them to the topology of the pre-generated network. The sequential section calculates the system's kinematics for 300 frames: at each time step, the displacement toward the Voronoi centroid is combined with the nonlinear attractive forces acting only on the nodes connected by the graph. The trajectories and partial computation times, recorded using the `chrono` library, are saved to a CSV file. Finally, the parallel section runs an iterative loop from 1 to 32 threads; at each iteration, the system is reset to its initial state and the kinematics are recalculated using OpenMP for both spatial tessellation and force calculation, exporting global performance logs for subsequent scalability analysis.

```
=====
2. AVVIO RUN PARALLELA CON OPENMP (1 -> 32 THREADS)
=====

-----
Thread OpenMP: 1 | Tempo Totale: 16.9172 s
1. Voronoi Esatto (OMP):      16.4536 s | 97.2598%
2. Posizioni Proposte:       0.0014 s | 0.0081%
3. Calcolo Forze (OMP):       0.1350 s | 0.0177%
4. Scrittura CSV & Update I/O: 0.3266 s | 1.9308%

-----

Thread OpenMP: 2 | Tempo Totale: 10.7982 s
1. Voronoi Esatto (OMP):       9.3518 s | 95.8660%
2. Posizioni Proposte:         0.0010 s | 0.0092%
3. Calcolo Forze (OMP):        0.0791 s | 0.0278%
4. Scrittura CSV & Update I/O: 0.3662 s | 3.3911%

-----
```

Figure 1: Esempio di stampa del ciclo parallelo

The visual post-processing module, implemented in `Visualizzazione.py`, handles the graphical repre-

sensation of the simulation. Within it, the function ‘voronoi_finite_polygons_2d’ addresses a native limitation of the SciPy library, which tends to generate open regions extending to infinity for agents located along the outer perimeter of the swarm. By intercepting the orthogonal vectors of the ridges lacking a terminal vertex, the function projects these coordinates onto a very high radius, allowing for the geometric reconstruction of closed and confined polygons. The dynamic generation of frames is instead handled by the update function, designed to interface with Matplotlib’s animation modules. In each frame, the function resets the Cartesian plane, redefines the limits of the 2D domain, and extracts the current coordinates of the agents. It then recalculates the Voronoi diagram, crops its extent within the physical boundaries of the area using Boolean intersection operations (provided by the Shapely library), and applies a pseudorandom color fill. The rendering concludes with the overlay of the magenta lines representing the adjacency matrix, the placement of the agent markers, and the insertion of their respective text IDs, before the final compilation into GIF format.

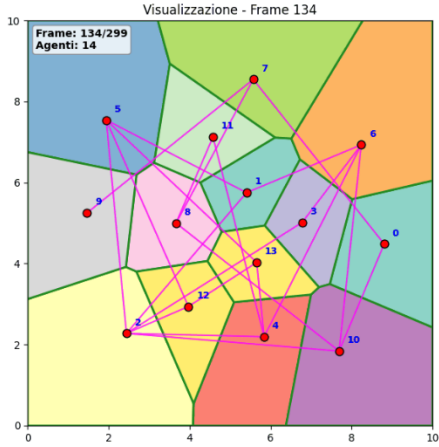


Figure 2: Esempio colorazione con 14 agenti

The ‘main’ function of the ‘generate_graphs.py’

script handles the quantitative performance analysis. The module parses the benchmark files generated by the C++ application and selects the relevant timing metrics, isolating the pure computation time. This value is obtained by summing only the times for Voronoi tessellation and the computation of inter-agent forces that is, the only components subject to parallelization. Finally, the function calculates the speedup and generates graphs showing execution times and scaling efficiency.

2.1 Performance

To verify the changes and the efficiency of parallelization, runs were performed with 100 and 500 agents, always over a 300-frame interval.

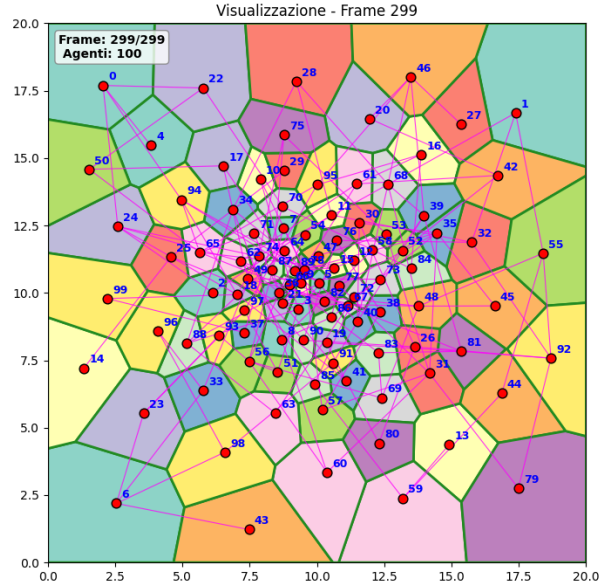


Figure 3: Frame finale simulazione con 100 agenti

The sequential baseline records a processing time of 759.71 ms (754.53 ms spent on Voronoi tessellation and 5.18 ms on force calculation). With the introduction of OpenMP, it is immediately apparent that the 1-thread parallel configuration is slightly slower than the sequential one (778.67 ms), a behavior likely due to the initial overhead of creating the thread pool. As

the number of threads increases, the time taken to execute the run progressively decreases until it reaches 15 threads, after which we observe a slight decline in performance.

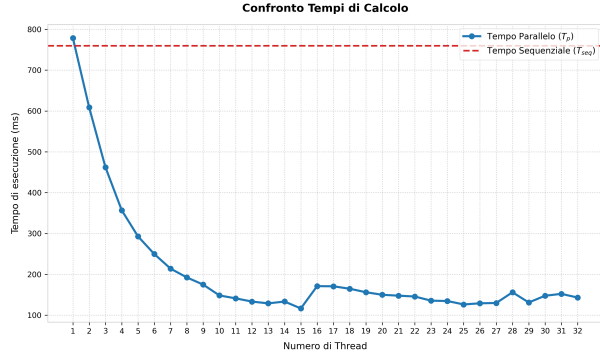


Figure 4: Confronto tempi run con 100 agenti

In fact, the best speedup is achieved at exactly 15 threads (6.51x): this is because the computer on which the script was run has 16 logical cores, so by allocating exactly 15 threads to the intensive computation, a single core is left available to the operating system (to handle everything else), and these threads can thus operate at full capacity without any interruptions. The bottleneck observed at 16 threads is due to context switching.

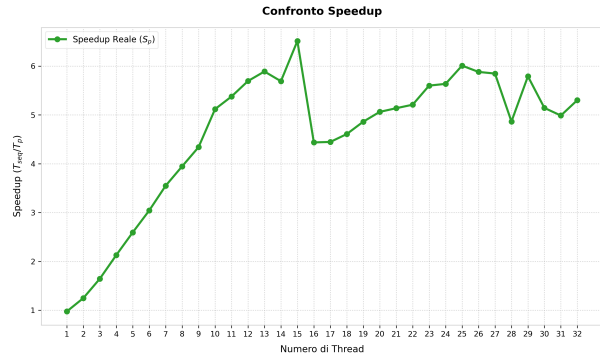


Figure 5: Grafico speedup con 100 agenti

The analysis of the scenario expanded to 500 agents confirms and amplifies the architectural dynamics

that emerged in the previous test. Due to the nature of the spatial partitioning algorithm, whose complexity grows non-linearly with the number of nodes, increasing the number of agents fivefold causes the computational load to skyrocket. Considering strictly the Voronoi time plus the force calculation, the sequential baseline in fact jumps from a modest 759.71 ms in the first test to a full 18,691.25 ms (of which 18,525.10 ms is dedicated solely to Voronoi and 166.15 ms to the forces). The time curve initially traces a very steep descent, halving rapidly in the early stages of parallelization. This nearly hyperbolic drop highlights how efficiently the architecture handles the initial load.

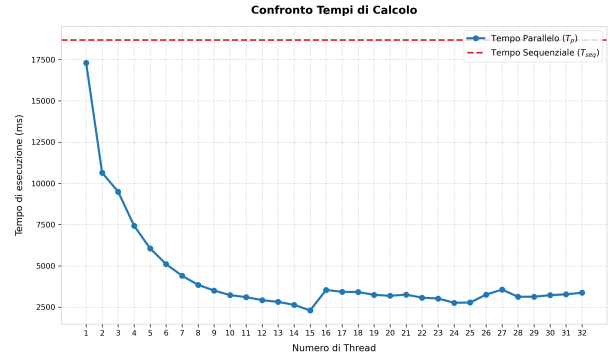


Figure 6: Confronto tempi run con 500 agenti

The most significant finding of this analysis is the jump in the maximum speedup, which rises from 6.51x, recorded with 100 agents, to 8.13x.

In this scenario as well, the point of maximum efficiency is reached at 15 threads, followed by a performance drop at 16 threads (where the time increases from approximately 2,298 ms to 3,536 ms).

The reason the speedup is significantly higher than in the case with 100 agents lies precisely in the balance between payload and overhead. In the previous test, the geometric problem was not very demanding and was solved in fractions of a second: the time required by the OpenMP construct to create threads, distribute the work, and rejoin them at the end of the cycle ended up having a massive impact on the total time. With 500 agents, on the other hand, the

computational load is so massive that the OpenMP overhead becomes entirely negligible. The CPU is finally able to dedicate nearly all its clock cycles to actual processing, allowing the parallel architecture to fully realize its potential and scale much more linearly and efficiently toward the physical limit of the machine.

computing, providing a robust and efficient architecture for simulating the complex dynamics of a swarm.

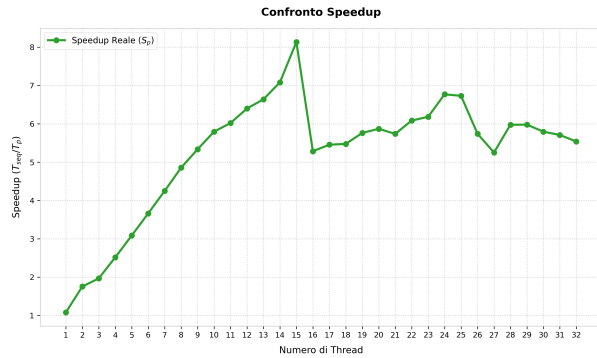


Figure 7: Grafico speedup con 500 agenti

3 Conclusions

This project successfully demonstrated the modeling and simulation of a multi-agent system, effectively combining the geometric constraints defined by Voronoi tessellation with physical dynamics driven by graph-based attractive forces. The transition from a purely sequential architecture to a parallel one using OpenMP provided important insights into the system's scalability dynamics. The analysis clearly demonstrated how the workload affects overall efficiency: while smaller simulations (100 agents) were quickly limited by thread management overhead, increasing the problem size to 500 agents allowed the parallel architecture to reach its full potential. In all scenarios, maximum efficiency was consistently observed at 15 threads. This result confirms that, when running such a computationally intensive application on a CPU with 16 logical threads, reserving one core for the operating system is a mandatory strategy to avoid the severe performance degradation caused by context switching. Ultimately, the project successfully combines computational geometry and parallel